

3

Red Team Infrastructure

Chapter Objectives

After reading this chapter and completing the exercises, you will be able to do the following:

- Understand common C2s
- Use C2 communication channels
- Use C2 redirectors
- Secure your infrastructure
- Understand how to set up C2 infrastructure

To begin setting up initial access, a red team infrastructure needs to be created. The infrastructure being used for a red team needs to blend in like legitimate traffic. Attention to detail during this step is critical because your infrastructure as an attacker is what sets you up for success while executing and maintaining access in a compromised environment.

In the last few years, there has been a significant shift toward using cloud platforms such as Azure and Google Cloud for red team infrastructure. Traditionally, operators would set up Command and Control (C2) infrastructure on self-hosted servers, but the move to cloud services allows for more flexible, scalable, and resilient infrastructures. The ability to dynamically spin up new instances or scale services based on the size of an engagement has reduced the operational overhead.

Additionally, cloud services often integrate with content delivery networks (CDNs), which can help blend C2 traffic with legitimate cloud provider traffic, adding an additional layer of obfuscation. CDNs such as Cloudflare or Azure's Classic CDN are increasingly used to route C2 traffic through legitimate and trusted web infrastructure. Red teamers can

abuse CDNs by blending in their traffic using these providers. This technique makes it more difficult for incident response teams to distinguish malicious traffic from legitimate traffic.

Another noticeable shift has been around the adoption of DNS over HTTPS for red teamers' C2 communication channels. These communications send DNS queries over HTTPS (DoH) connections, thus bypassing traditional network monitoring systems. It is critical to note that while DoH provides obfuscation, it also introduces challenges in ensuring stable and reliable communication because some corporate networks might block nonstandard DNS requests. Additionally, older C2 frameworks leveraged static callback intervals that could be detected by network monitoring systems. The introduction of jitter in C2 profiles opened up the door for red teamers to randomize their traffic associated with their agents and avoid predictable intervals.

The purpose of this chapter is to review how development of infrastructure has shifted in the past few years, reviewing common C2 communication channels, and how you can set up your infrastructure for success. When you're preparing your red team infrastructure, you might want to set up some items versus other items that may not be needed, depending on the objective of your engagement. Red teams can operate from an assumed breach standpoint or start from the perimeter, which requires targeting an external application or user. We will dive deeper into the methodology side of red teaming in [**Chapter 4, "Modern Red Teaming Methodology and Tools."**](#) Additionally, if social engineering is involved, you will need to prepare phishing infrastructure or vishing infrastructure, which we will discuss in [**Chapter 5, "Social Engineering and Physical Assessments."**](#)

An Overview of Red Team Infrastructure

In years past, red team infrastructure for Command and Control communications didn't need to be as sophisticated as needed today; however, there has been a shift in recent years. For example, leveraging CDNs makes red teamers' traffic appear to be from a legitimate company such as Microsoft or other cloud providers. The traffic from major cloud providers is typically trusted because real companies utilize CDNs for

their infrastructure. This technique circumvents traffic filters a company might have in place on the perimeter if using CDNs.

The infrastructure for a red team is the heart of your engagement and how your operation comes together as a whole. Different elements should be considered when building your infrastructure, including what is required for meeting your objective, securing your infrastructure, having backup infrastructure in place, and ensuring proper logging throughout the engagement lifecycle for reporting purposes.

Command and Control

Command and Control servers are what operators use to manage their operation to remotely perform actions on victims' computers. These servers are dependent on the C2 type being used because there are tons of commercial and open-source options. The C2 matrix is an available open-source document that professionals in the field can reference to understand every C2 available from a commercial and public perspective with information on agent types, socks support, and so on. The information is located at <https://howto.thec2matrix.com>.

The top C2 options in the field from a commercial standpoint are Nighthawk by MDSec and Cobalt Strike by Forta. The most common publicly available C2 frameworks supported by the community are Mythic, Sliver, Havoc, and BadRats. An advantage of public C2 frameworks is the flexibility for customization to be most effective, whereas commercial C2s can be more restrictive in terms of customization. Commercial C2s are still signed in the wild by EDRs when default options are in use but typically require less customization than a public C2 option out of the box.

Nighthawk (<https://www.mdsec.co.uk/nighthawk/>) was released in 2021 and is the latest advanced tradecraft C2 framework; it is intended for use only by authorized red team operators. The MDSec team built the C2 by leveraging custom research and development, which empowers the advanced capabilities of the C2. The C2 supports multiple operators, advanced evasion techniques, custom loaders, malleable profiles, SOCKS5,

Beacon Object Files (BOFs), covert communications, secure post exploitation options, and a custom PE generator for operators.

Cobalt Strike (<https://www.cobaltstrike.com/>) was originally created by Raphael Mudge in 2012 as a commercially available C2 for operators to leverage during red team operations. Cobalt Strike comes with an Arsenal Kit that is a tool suite of custom tools to improve your operator capabilities for a successful engagement; it includes a sleep mask kit, custom loaders, and other options to choose from. Cobalt Strike is widely used by cybersecurity firms in the community for red team operations. The C2 also supports multiple operators, BOFs, malleable profiles, SOCKS4/SOCKS5, and covert communications.

Mythic (<https://docs.mythic-c2.net/>) is intended to serve as a plug-and-play architecture where operators have the ability to create changes to communication channels, agents, and other aspects as they operate. This C2 framework is the only one supporting Linux, Mac, and Windows implants that provides flexibility for operators. Additionally, Mythic supports socks and secure communications. Mythic is made up of a web-based frontend utilizing React and Docker contained for the backend of the C2. Another great feature with Mythic is the ability to map tasks executed during an engagement to the MITRE (<https://attack.mitre.org>) framework.

The MITRE Framework

MITRE ATT&CK, also known as MITRE Adversarial Tactics, Techniques, and Common Knowledge, is a publicly available knowledge base of threat actor behaviors. These behaviors are mapped to specific tactics, techniques, and procedures (TTPs) that any organization can utilize to have a common language between offensive and defensive security teams. The tactic areas include reconnaissance, resource development, initial access, execution, persistence, privilege escalation, defense evasion, credential access, discovery, lateral movement, collection, C2, exfiltration, and impact in the ATT&CK matrix. The tactics areas are also broken out into 191 techniques and 385 subtechniques. Operators can leverage the framework for ATT&CK mappings to refine their C2 communications strategies

by understanding what TTPs will likely get picked up in an operation from previous adversaries and how they can fill those gaps to go under the radar.

Mythic has various community profiles

(<https://github.com/MythicC2Profiles>) to leverage with the framework, including the following:

- **http**: Utilizes GET/POST requests to the Mythic server supporting kill dates, sleep intervals, custom headers, proxy information, and SSL support.
- **dns**: Leverages a DNS server listening for TXT queries from the agent for communications supporting kill dates, sleep intervals, and a custom DNS server.
- **httpx**: Is a dynamic and customizable HTTP profile. There is a range of options that an operator can leverage, including callback domains, domain fallback, domain rotation, encrypted key exchange, jitter, sleep intervals, and agent messages.
- **discord**: Leverages Discord's REST API for communications.
- **websocket**: Allows agents to connect through a static websocket connection with support for SSL, kill dates, and sleep intervals.
- **dynamichttp**: Employs extendable HTTP messages with dynamic modifications for the client and server supporting kill dates, sleep intervals, custom headers, proxy information, and SSL support.

C2 Communication Channels

Command and Control communications channels are required for an operator to communicate with the C2 server. These channels are used to send data, receive data, and maintain persistent access to an agent on a victim's machine. The communication methods are just as important as your evasion tactics when delivering your payload. If you have poor operational security (OPSEC) for your C2 channels, you will likely get caught.

As blue teams have evolved over time, operators have had to make their traffic blend in with existing network traffic. The most common C2 com-

munications methods today are DNS over HTTPs and HTTP/HTTPS, although HTTPS seems to be the top method overall.

- **DNS over HTTPS (DoH)** provides encryption end-to-end similarly to using HTTPS. The DNS queries and DNS responses are sent over HTTPS. Operators can abuse this method to reach out to DNS providers specified by the operator, such as cloudflare-dns.com, and resolve the DNS queries all over an encrypted channel.
- **HTTP/HTTPS** is legitimately used for web traffic across the Internet. Most if not all networks will allow HTTP/HTTPS protocol communication, which is a benefit for operators to disguise their traffic using these protocols. From an OPSEC perspective, it is best to not use HTTP in general because the traffic will be unencrypted. The safest way to have your traffic blend into an environment is to ensure that it is encrypted.

Redirectors

Redirectors relay traffic incoming from an external standpoint and pass the traffic to the location specified, which could be to a host that is only reachable through the initial redirector. Redirectors are used for C2 traffic to hide the C2 server's location, deceive the traffic to look legitimate, and also protect your C2 server from being burned. The beauty of redirectors is that even if one of your redirectors gets burned, your C2 server is safe, and you can replace the external redirector with a new one without needing to reset any of your C2 infrastructure.

Redirectors that will take less effort such as **socat** or **iptables** will blindly forward traffic to your C2 server, which can be a downfall from an operational security standpoint because Internet scanners may be able to identify the malicious traffic. However, using more sophisticated redirectors that leverage reverse proxies with rules helps filter down only the traffic you want passed by leveraging specific user agents, allowing only certain source IPs or other specific conditions to protect the infrastructure.

Redirector Types

The following are redirector types, beginning with basic methods to more sophisticated methods:

- **socat** is a client used to blindly forward all incoming connections, known as “dumb pipe” traffic. The tool establishes bidirectional streams between the specific IP and port.
- **iptables** is typically used as a firewall tool to control network traffic and is a native component for Linux networking. With **iptables**, you can have multiple rules and configurations for your port forwarding.
- **Reverse proxies** are used to protect your C2 backend by leveraging Apache or Nginx web server configurations to forward your traffic to the next web server specified. This helps protect the backend server’s identity and leverage SSL certificates from an external point of view.
- **Content delivery networks** (CDNs) are a group of servers used to geographically distribute content from your site to provide faster performance for an end user. You can abuse CDNs by leveraging their distribution addresses that will be in front of your domain. CDNs are typically used alongside a reverse proxy setup and filter the web servers to allow only the CDNs’ source IPs to access the external redirector.
- **CDNs with reverse proxy using rewrites** combine a standard reverse proxy web server with a CDN in front of the web server and take into account rewrites specific in the Apache/Nginx configuration. These rewrites can control the directive and manipulate the URL on the fly based off the configurations in place.

Command and Control Setup

In this section, you will learn how to set up and execute operations with the Mythic C2. [Figure 3-1](#) shows an example of building out infrastructure for a typical red team.

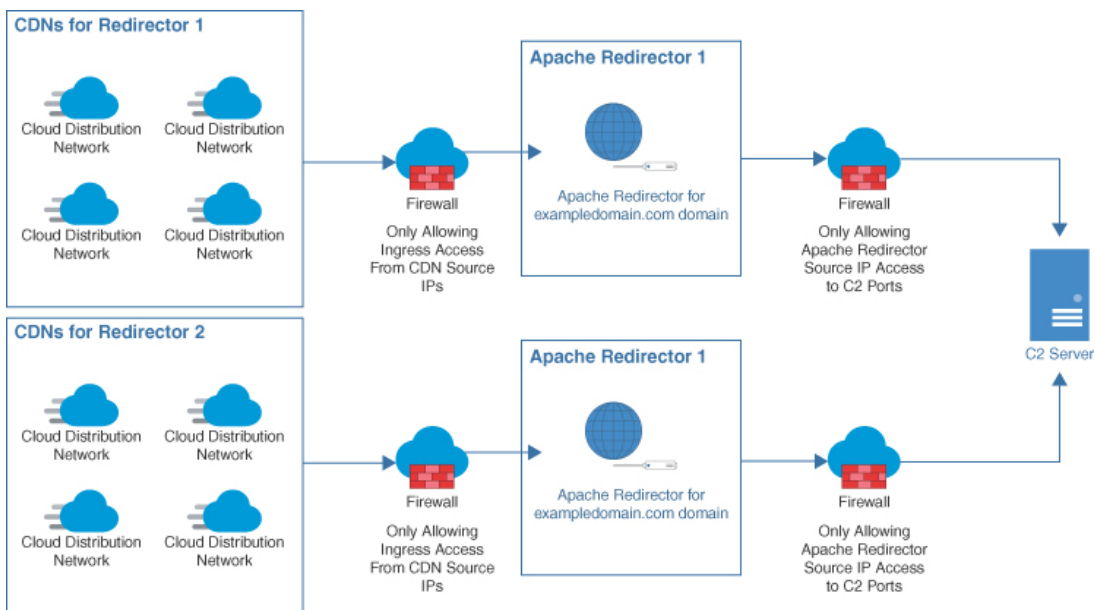


Figure 3-1

C2 Infrastructure Setup

To get started, you will need the following setup:

- Azure Ubuntu Server (B1s) to serve as the redirector
- Azure Ubuntu Server (B2ms) to serve as the C2 endpoint
- Azure CDNs for redirector domain
- Operator box (virtual machine or cloud instance)
- Windows testing box (defender disabled)

Once it is deployed, we will provision the Apache server for the domain being used for the redirector. A blog post from Optiv's Attack & Penetration team (archived at

https://lazzslayer.github.io/References/Redirectors/Redirectors_Optiv.pdf)

provides easy-to-use Apache configs that you can leverage for your server for Apache's site configuration. You will also need to set up the A records for your domains being used for the infrastructure. For SSL certificate generation, you will also need to *temporarily* open 80/443 to the world to retrieve certificates for the domain, and then you can filter it back down to allow only Azure CDN source IPs.

Apache Redirector

Examples 3-1 through **3-4** walk through how to set up your Apache server for your redirector from installing to configuring the server.

Example 3-1 Installing Apache

[Click here to view code image](#)

```
sudo apt update
sudo apt full-upgrade
apt-get install apache2
systemctl stop apache2.service
a2enmod ssl proxy remoteip rewrite proxy_http
```

Example 3-2 Installing Certificates for Your Domain

[Click here to view code image](#)

```
apt-get install certbot

certbot certonly --standalone --preferred-challenges http -d exampledomain.com
--register-unsafely-without-email --agree-tos
```

Example 3-3 Apache Config for Redirector

[Click here to view code image](#)

```
<VirtualHost *:80>
ServerName exampledomain.com
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]

ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
<VirtualHost *:443>
ServerName exampledomain.com
RemoteIPHeader X-Forwarded-For

ProxyPass / https://exampledomain.com/
ProxyPassReverse / https://exampledomain.com/
```

```
SSLProxyEngine On
SSLProxyVerify none
SSLProxyProtocol TLSv1.2

SSLCertificateFile /etc/letsencrypt/live/exampledomain.com/fullchain.pem
SSLCertificateKeyFile /etc/letsencrypt/live/exampledomain.com/privkey.pem

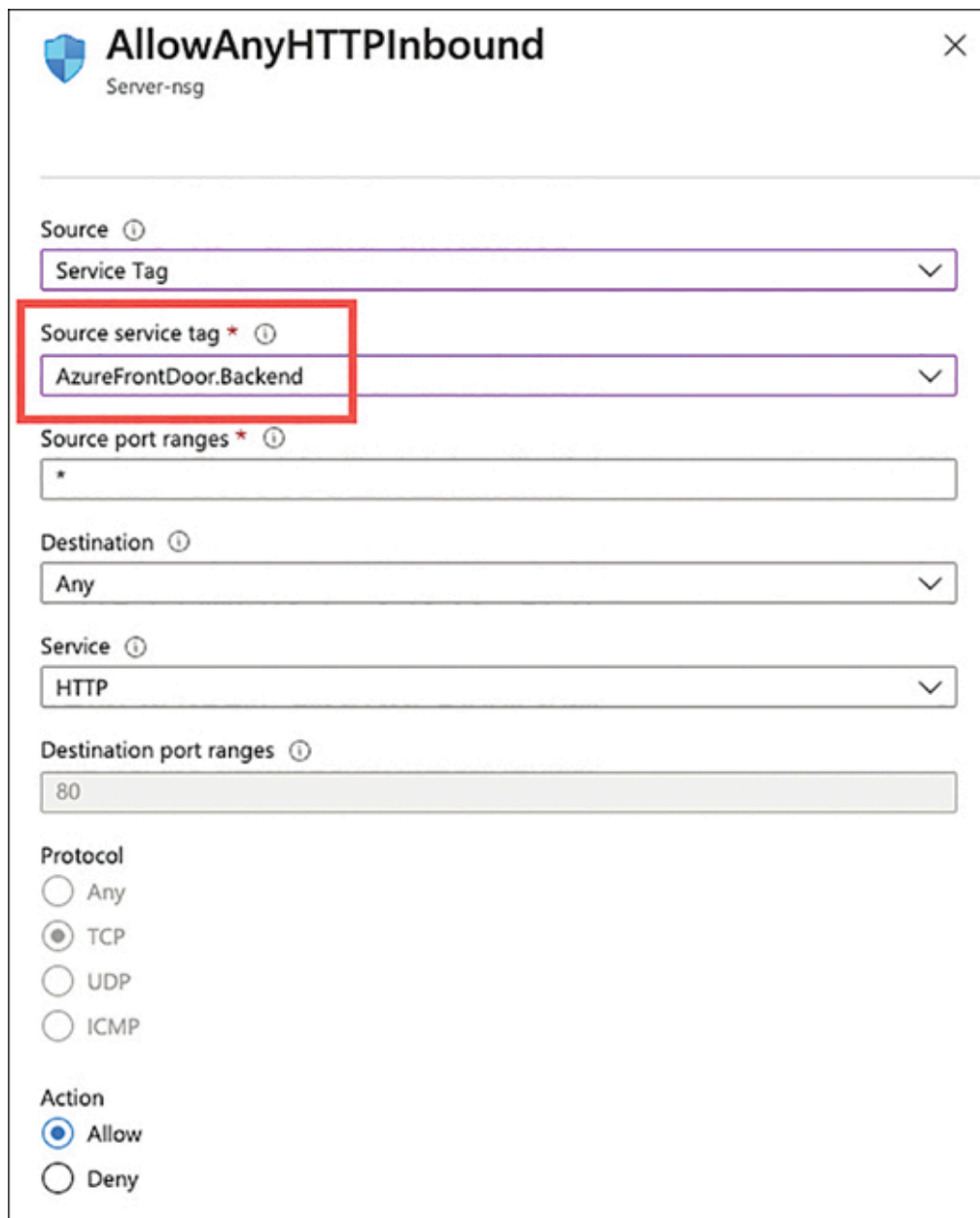
ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

Example 3-4 Adding in the IP of Your C2 Server in /etc/hosts

[Click here to view code image](#)

```
echo <C2ServerIP> <C2Domain> >> /etc/hosts
```

Now that your redirector is set up, you can configure your redirector to allow only Azure's CDN from HTTP and HTTPS using the Azure service tag—AzureFrontDoor.Backend. **Figure 3-2** shows how to update it for HTTP, but you must ensure that you update the rules for both HTTP and HTTPS.



AllowAnyHTTPInbound
Server-nsg

Source ⓘ
Service Tag

Source service tag * ⓘ
AzureFrontDoor.Backend

Source port ranges * ⓘ
*

Destination ⓘ
Any

Service ⓘ
HTTP

Destination port ranges ⓘ
80

Protocol
☐ Any
☒ TCP
☐ UDP
☐ ICMP

Action
☒ Allow
☐ Deny

Figure 3-2

Filtering HTTP

Next, you will create a CDN for your redirector domain with Azure Classic CDN, as demonstrated in **Figure 3-3**.

Add an endpoint

Allows configuring content delivery behavior and access.

Name *
exampleforbook ✓
.azureedge.net

Origin type *
Custom origin ✓

Origin hostname * ⓘ
yourdomain.com ✓

Origin path ⓘ
/Path

Origin host header ⓘ
yourdomain.com ✓

☒ HTTP port ⓘ
*
80

☒ HTTPS port ⓘ
*
443

Optimized for ⓘ
General web delivery ✓

Figure 3-3

Creating CDN Endpoint

Next, you will configure the compression and caching settings on your CDN endpoint, as illustrated in **Figure 3-4**. You should disable both of these settings to avoid any issues with communications to your end server.

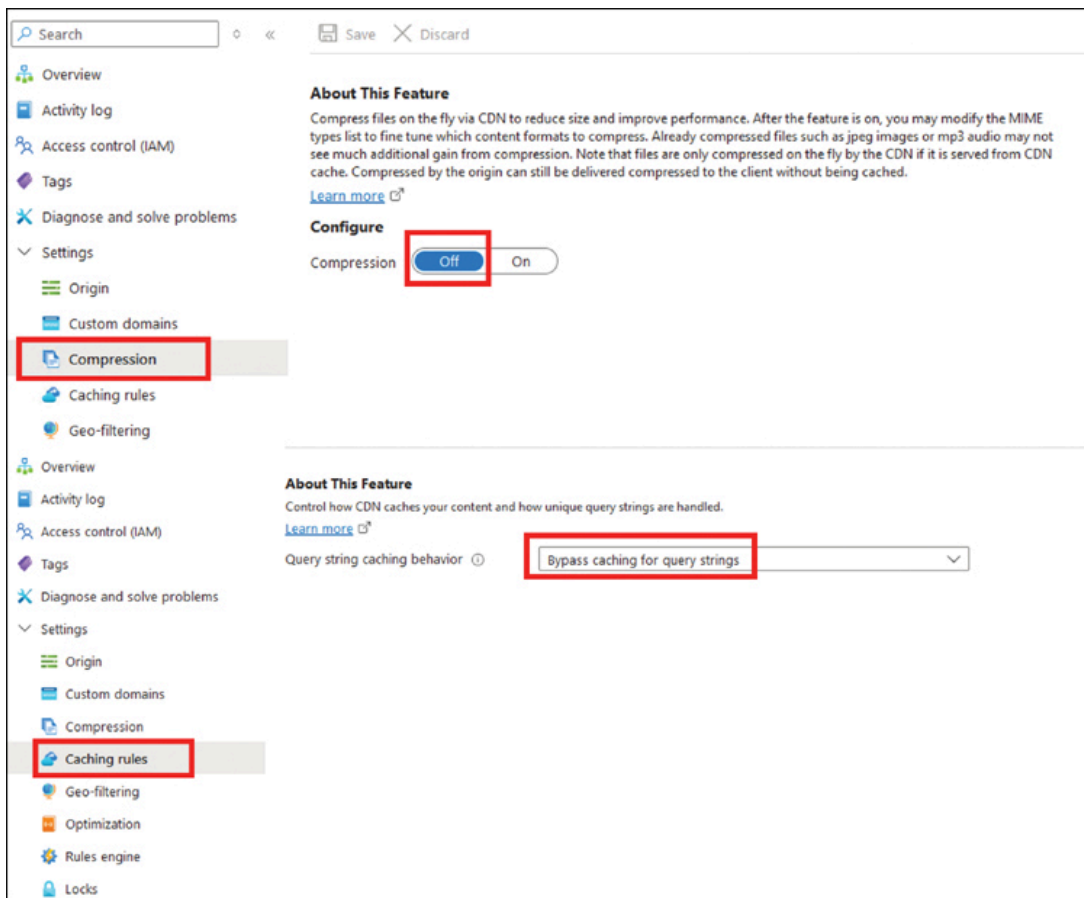


Figure 3-4

Compression and Caching Configurations

C2 Server

Prior to setting up Mythic, you should ensure that the A record for the domain being used points to the C2 IP. For SSL certificate generation, you will also need to *temporarily* open 80/443 to the world to retrieve certificates for the domain. You will change the firewall rules after this step to allow traffic only from your redirector IP.

Example 3-5 demonstrates how to install certificates from Let's Encrypt for your domain on your C2 server and move them into your Mythic directory. The \$MYTHICDIRECTORY should be set to where your Mythic directory is stored on your C2 server.

Example 3-5 Installing Certificates for Your Domain

[Click here to view code image](#)

```
MYTHICDIRECTORY=(/opt/Mythic)
sudo apt-get install certbot
certbot certonly --standalone --preferred-challenges http -d
    exampledomain.com --register-unsafely-without-email --agree-tos
cp /etc/letsencrypt/live/exampledomain.com/privkey.pem $MYTHICDIRECTORY
cp /etc/letsencrypt/live/exampledomain.com/fullchain.pem $MYTHICDIRECTORY
```

After the certificates are generated, lock down your instance to allow only your redirector's IP access via HTTPS (443). Taking this step will prevent scanners from directly reaching your C2. If the cloud instance where your C2 is hosted gets identified as malicious and is reachable from the Internet, you risk your instance being taken down by the cloud provider, along with scanners lighting up while scanning your host.

Additionally, you can now remove your A record from your C2 domain because the change in [Example 3-5](#) will show the redirector server where the C2 domain points to.

Next, you can install Mythic by using the steps shown in [Example 3-6](#).

Example 3-6 Installing Mythic

[Click here to view code image](#)

```
git clone https://github.com/its-a-feature/Mythic
cd /Mythic
./install_docker_ubuntu.sh
```

Out of the box, Mythic does not come with any C2 profiles or agents. To install specific agents and profiles, you can use the various options available at the following locations:

- **Mythic Agents:** <https://github.com/MythicAgents>
- **Mythic C2 Profiles:** <https://github.com/MythicC2Profiles>

For this walkthrough, you can install the http C2 profile and the Apollo agent, as shown in [Example 3-7](#). We will leverage the HTTP profile be-

cause it operates on the same protocol as the Mythic server rather than the other profiles for the purpose of this walkthrough.

The HTTP profile includes the ability to set kill dates and sleep intervals, as well as customize the headers, proxy information, and SSL support. The Apollo agent is .NET based with features for dynamic command loading, SOCKS5 support, peer-to-peer communication, unmanaged code execution, and tons of other useful features for an operator. The Apollo agent is also used in SpecterOps (<https://specterops.io/>) annual trainings.

[Example 3-7](#) and [Example 3-8](#) demonstrate how to install your profile and agent and then start the Mythic server.

Example 3-7 Installing the C2 Profile and Agent

[Click here to view code image](#)

```
sudo ./mythic-cli install github https://github.com/MythicAgents/Apollo.git
sudo ./mythic-cli install github https://github.com/MythicC2Profiles/http
```

Example 3-8 Starting Mythic

```
# Start Mythic Client
sudo -E ./mythic-cli start
```

To access the Mythic GUI, you can do SSH local port forwarding from your operator box with the command demonstrated in [Example 3-9](#).

Tip

It is recommended that you operate from an operator box for each engagement as you conduct operations. An Ubuntu virtual machine or cloud instance will suffice for this purpose.

Example 3-9 Accessing the GUI

[Click here to view code image](#)


```
# SSH Port Forwarding
```

```
ssh -L 7443:127.0.0.1:7443 user@mythicserver
```

Once the forwarding is in effect, navigate to <https://127.0.0.1:7443> to log in (see [Figure 3-5](#)). You can find the mythic_admin password in the `.env` file of the Mythic directory.

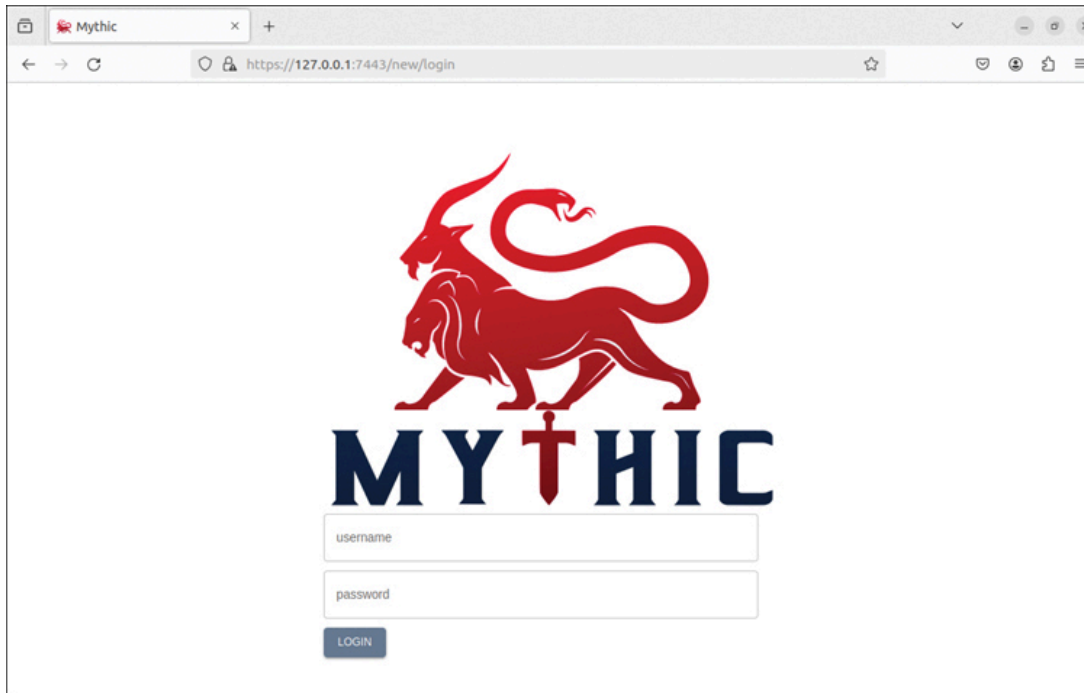


Figure 3-5

Mythic Login

After you are logged in, navigate to the **Agents & C2** section on the left panel, as illustrated in [Figure 3-6](#). You will need to edit the C2 profile to include the certificates you previously generated.

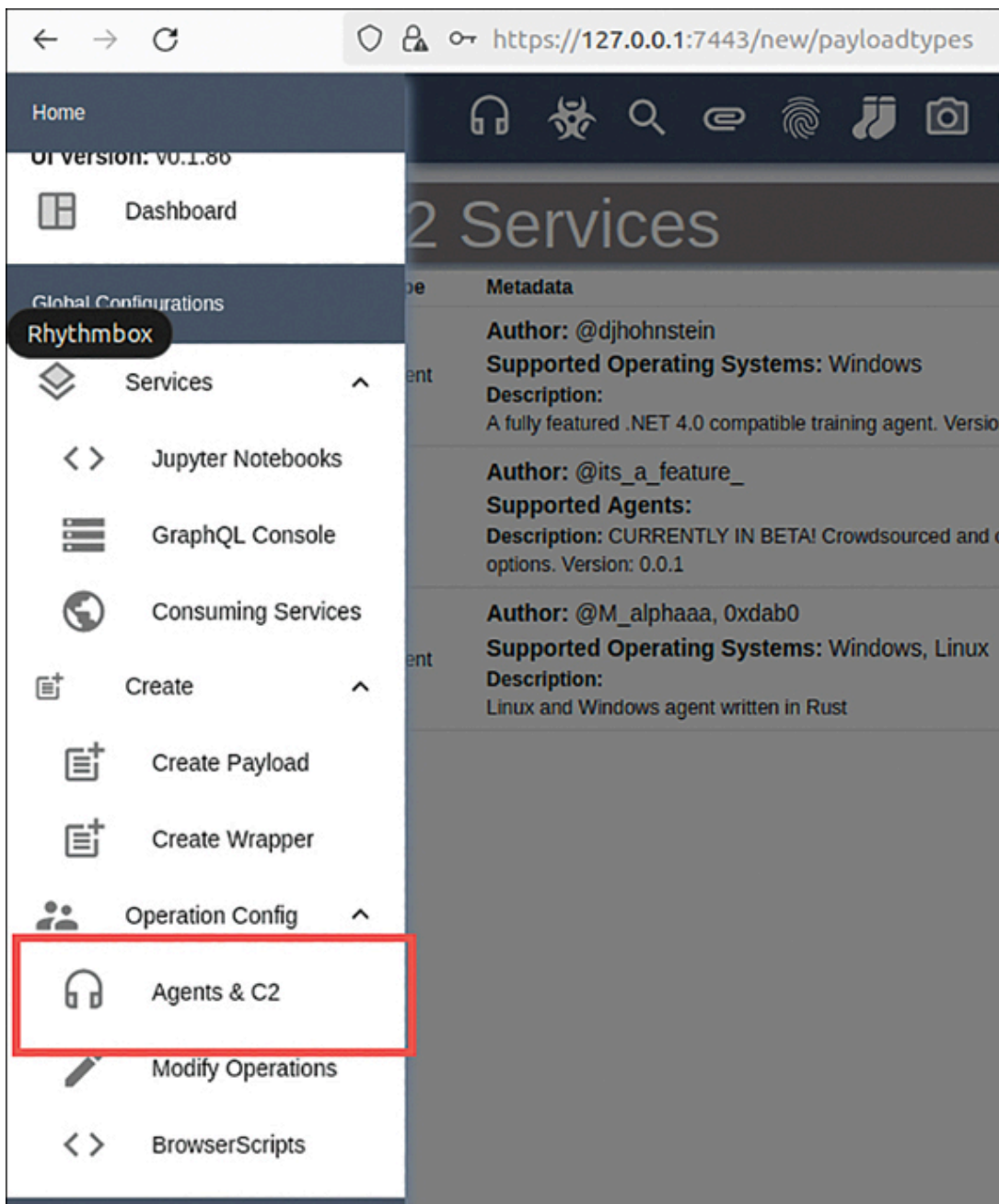


Figure 3-6

Updating the C2 Profile Config

Prior to editing the profile configuration, you will need to update the values for the profile by clicking the Clipboard icon and editing the fullchain.pem and privkey.pem files and adding in the domain certificates generated from [Example 3-5](#).

Simply copy the value from /etc/letsencrypt/live/yourdomain.com/fullchain.pem and add it into the fullchain.pem file. The same step can be done for the privkey.pem from /etc/letsencrypt/live/yourdomain.com/privkey.pem. [Figure 3-7](#) illustrates where you can find those files in the C2 configuration.

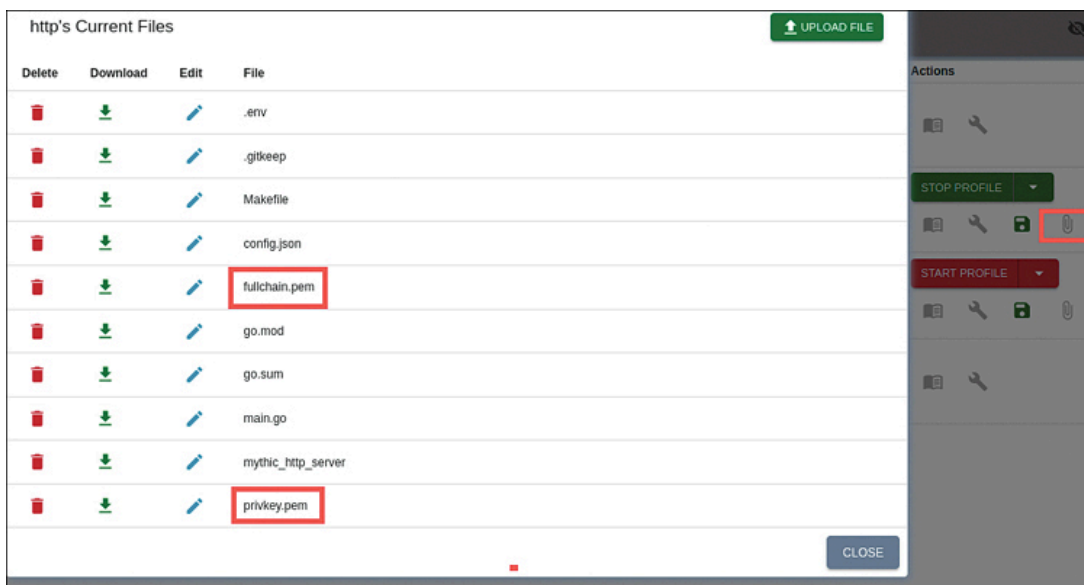


Figure 3-7

Updating Certificate Files for the C2 Config

Next, you will edit the config to ensure that it does not use the default options. You can customize this part as you want, as illustrated in **Figure 3-8**. For this example, you can use similar headers as Microsoft-IIS/10.0, as demonstrated in **Example 3-10**.

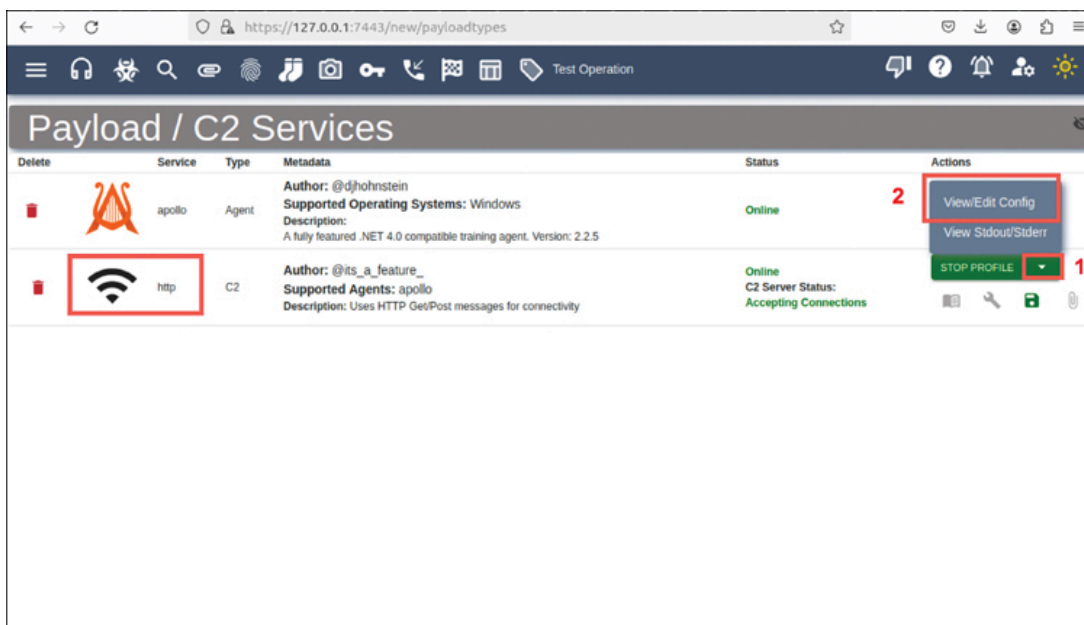


Figure 3-8

Editing the Config

Example 3-10 http Config

[Click here to view code image](#)

```

{
  "instances": [
    {
      "ServerHeaders": {
        "Server": "Microsoft-IIS/10.0",
        "Cache-Control": "max-age=0, no-cache",
        "Pragma": "no-cache",
        "Connection": "keep-alive",
        "Content-Type": "text/html; charset=utf-8"
      },
      "port": 443,
      "key_path": "privkey.pem",
      "cert_path": "fullchain.pem",
      "debug": false,
      "use_ssl": true
    }
  ]
}

```

Next, you need to stop and start your profile to ensure the changes went into effect (see [Figure 3-9](#)).

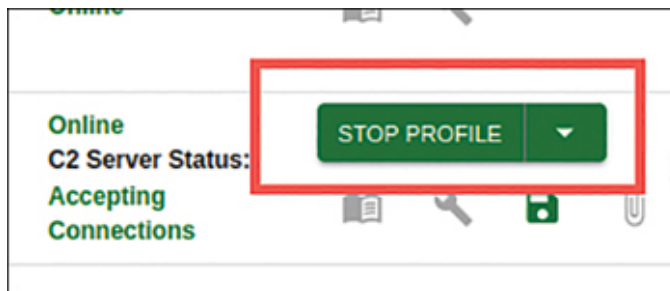


Figure 3-9

Starting and Stopping the Profile

Then you can test your traffic flow by hitting your CDN endpoint. In this case, you should receive a 405 response, which indicates the traffic is passing through properly because you are not specifying a specific endpoint on the server; however, it shows the traffic hits end to end, as demonstrated in [Example 3-11](#).

Example 3-11 Checking the Connection from End to End

[Click here to view code image](#)

```
curl -I https://exampleplaceholder.azureedge.net
HTTP/2 405
date: Sun, 23 Jun 2024 20:20:25 GMT
content-type: text/plain
content-length: 22
x-azure-ref: <snipped>
x-fd-int-roxy-purgeid: 0
x-cache: TCP_MISS
```

Now, you can generate payloads for Mythic by using the Apollo agent. Select the Windows option for the Target Operating System, as illustrated in [Figure 3-10](#).

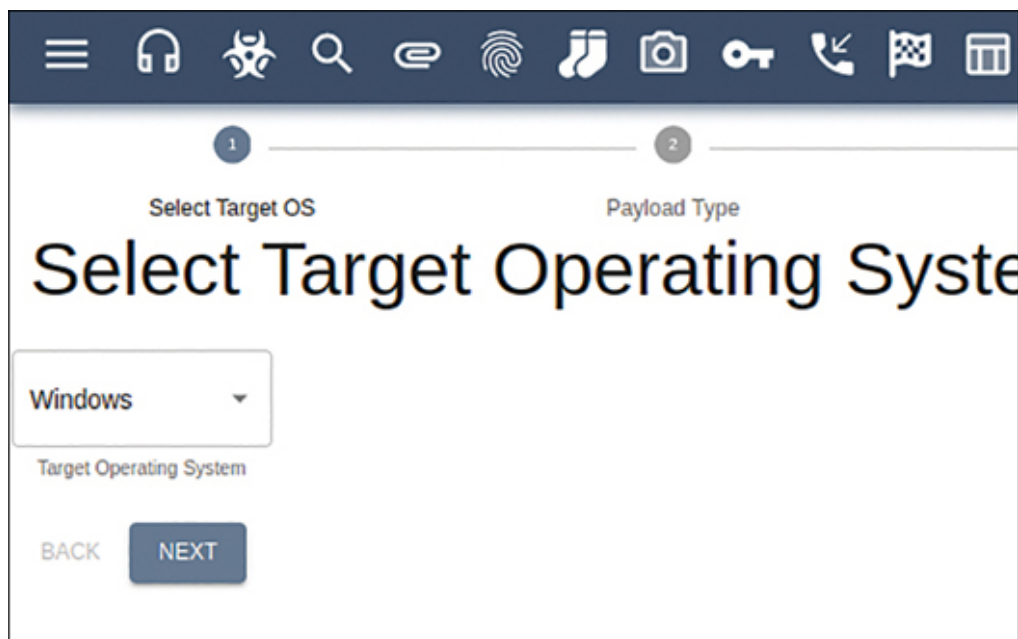
The screenshot shows the Mythic Apollo agent web interface. At the top, there is a dark blue header bar with various icons. Below the header, there are two numbered steps: '1 Select Target OS' and '2 Payload Type'. The main heading is 'Select Target Operating System'. Below this, there is a dropdown menu with 'Windows' selected. Underneath the dropdown, it says 'Target Operating System'. At the bottom, there are two buttons: 'BACK' and 'NEXT'.

Figure 3-10

Selecting the Target OS

The Target Payload Type will be *apollo* and the output will be *WinExe*, as illustrated in [Figure 3-11](#).

1 Select Target OS 2 Payload Type 3 Select Command

Select Target Payload Type

apollo ▾

Build Parameters

Build Parameter	Value
Output as shellcode, executable, or dynamically loaded library.	WinExe ▾

BACK NEXT

Figure 3-11

Output Option–Executable

Ultimately, an operator determines which commands are built into the agent and can add or remove which ones are included in the payload (see [Figure 3-12](#)).

1 Select Target OS 2 Payload Type 3 Select Commands 4 Select C2 Profiles 5 Build

Build Commands Into Agent

☐ sc
 ☐ screenshot
 ☐ screenshot_inject
 ☐ set_injection_technique
 ☐ steal_token
 ☐ unlink
 ☐ whoami

>>
 >
 <
 <<

Commands Included

☐ download
 ☐ exe
 ☐ load
 ☐ ps
 ☐ run
 ☐ shell

SC DOCUMENTATION

Commandline Help: sc
 Needs Admin Permissions: False
 Description: Service control manager wrapper function

BACK NEXT

Figure 3-12

Commands Built Into Payload

The C2 Profile section is customizable for the GET and POST requests. The URIs can be customized to common requests similar to IIS servers or to emulate another type of traffic to blend into the network. *Do not use the default options in this C2 profile section.*

There are multiple changes you will be making in this section, as illustrated in **Figure 3-13**:

- **Callback Host:** The host for your traffic to call back to externally. This will be the first hop from an external perspective.
- **Callback Interval:** How often your agent will call back to you in seconds; this number is in fixed intervals.
- **Callback Jitter:** The jitter percentage for the callbacks of the agent; this adds randomness to the callback to make each agent check-in vary.
- **Callback Port:** The port associated with your host that will be called back to.
- **Encryption Type:** The encryption method used for the traffic.
- **Headers:** Customization of the GET/POST messages.
- **Kill Date:** End date for your agent to call back.

Select C2 Profiles			
Include?	C2 Name	Pre-created Instances	Description
☒	http		Uses HTTP Get/Post messages for connectivity
Parameter	Value		
Callback Host Modified	https://[REDACTED].zureedge.net		
Callback Interval in seconds Modified	30		
Callback Jitter in percent Modified	20		
Callback Port Modified	443		

Figure 3-13

Modifying C2 Profile Options

The Payload Review section allows an operator to customize the final payload name and add a description for the payload, as illustrated in **Figure 3-14**. The description option is useful for making specific payloads for specific targets during an operation.

Select Target OS Payload Type Select Co

Payload Review

testing.exe

description

BACK CREATE PAYLOAD START OVER CREATE WRAPPER

Figure 3-14

Customizing the Output Filename

All generated payloads for the operation you are currently using on Mythic are available at <https://127.0.0.1:7443/new/payloads>, as illustrated in [Figure 3-15](#).

Delete	Modify	Build Progress	Download	File	Description
	ACTIONS			testing.exe	Created by mythic_admin at 2024-06-16 22:33:03 Z

Figure 3-15

Payloads Page

You can download the payload locally to your machine and transfer it over to your testing Windows box, as illustrated in [Figure 3-16](#) and [Figure 3-17](#).

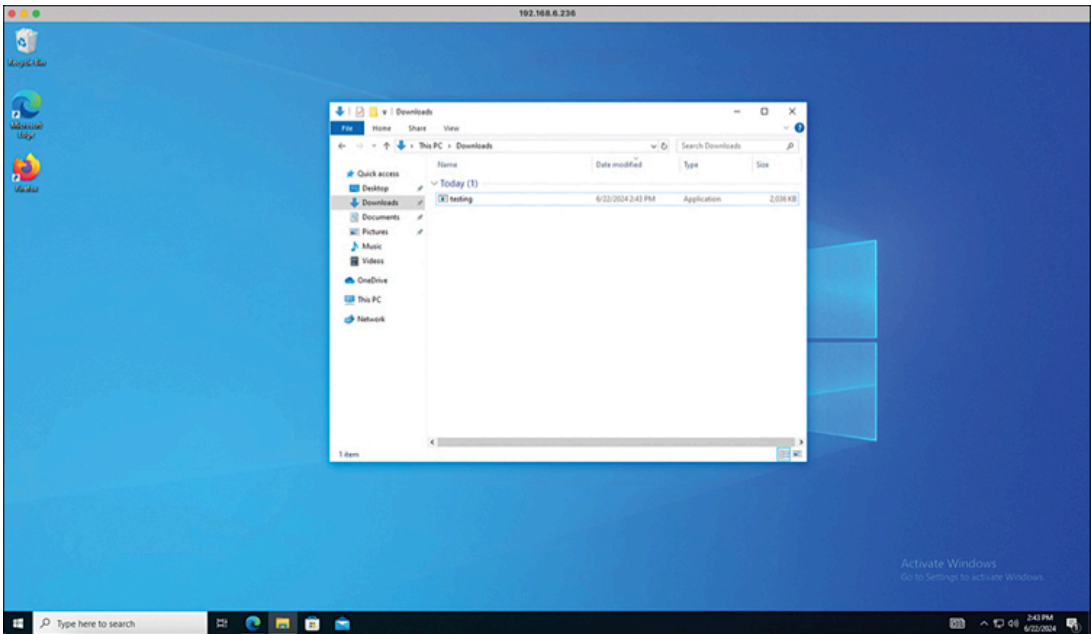


Figure 3-16

Transferred testing.exe Payload on the Windows Box

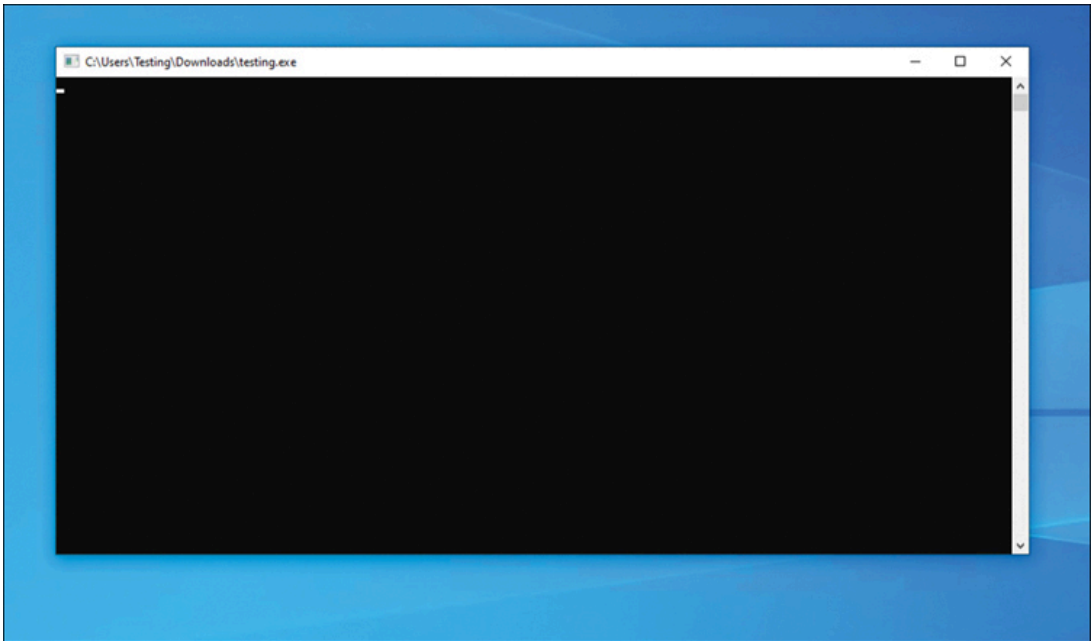


Figure 3-17

Executing testing.exe on the Windows Box

Upon execution, an agent has called back to the C2 server, as illustrated in **Figure 3-18**.

INTERACT	IP	HOST	USER	DOMAIN	PID	LAST CHECKIN	DESCRIPTION
5	192.168.6.236		Testing		2252	34 seconds	Created by mythic_admin at

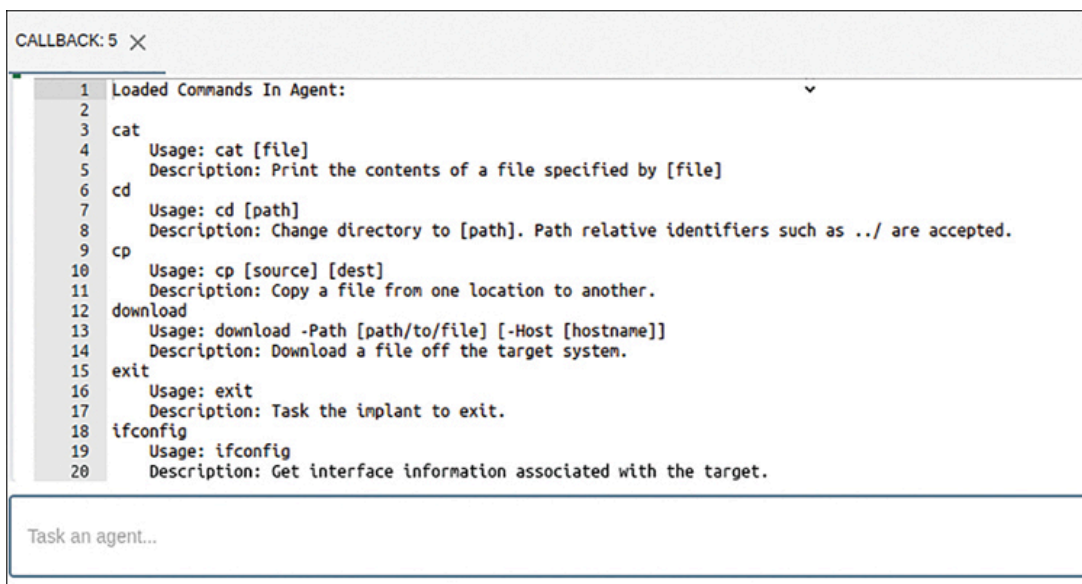
Figure 3-18

Mythic Callback

Note

In a real scenario, you would need to alter the C2 agent because it's heavily signed and also would need to make additional modifications to obfuscate your shellcode for delivery to bypass protections in place for an endpoint. We will discuss potential options to obfuscate your shellcode in [Chapter 4](#).

In the Mythic agent, click the **keyboard** option to interact with the agent. Then type **help** in the console to see which command options the agent has the ability to use on the victim host, as demonstrated in [Figure 3-19](#). Feel free to try different command options at this point and take into account which may be safer from an OPSEC perspective and which commands are better to stay away from for an operation.



```
CALLBACK: 5 X
1 Loaded Commands In Agent:
2
3 cat
4   Usage: cat [file]
5   Description: Print the contents of a file specified by [file]
6 cd
7   Usage: cd [path]
8   Description: Change directory to [path]. Path relative identifiers such as ../ are accepted.
9 cp
10  Usage: cp [source] [dest]
11  Description: Copy a file from one location to another.
12 download
13  Usage: download -Path [path/to/file] [-Host [hostname]]
14  Description: Download a file off the target system.
15 exit
16  Usage: exit
17  Description: Task the implant to exit.
18 ifconfig
19  Usage: ifconfig
20  Description: Get interface information associated with the target.

Task an agent...
```

Figure 3-19

Mythic help Output

The **socks** option will be the most useful option for an operator gaining access on a victim host because it allows the operator to leverage the victim's internal connection and route traffic through the victim's host via

a SOCKS5 proxy. [Example 3-12](#) demonstrates how to install proxychains4 on the server, start the socks server, and then adjust the sleep and jitter for the agent.

Example 3-12 Setup Proxy

[Click here to view code image](#)

```
# On the C2 Server, install proxychains4
apt-get install proxychains4

# On the C2 Server, edit /etc/proxychains4.conf
socks5 127.0.0.1 7006

# In the Mythic console, start the SOCKS5 proxy on Mythic agent
socks 7006

# In the Mythic console, set the agent to interactively callback with sleep
sleep 0 0
```

You can find the SOCKS proxies by clicking the Socks icon (see [Figure 3-20](#)); they also are available at <https://127.0.0.1:7443/new/search?tab=socks>.

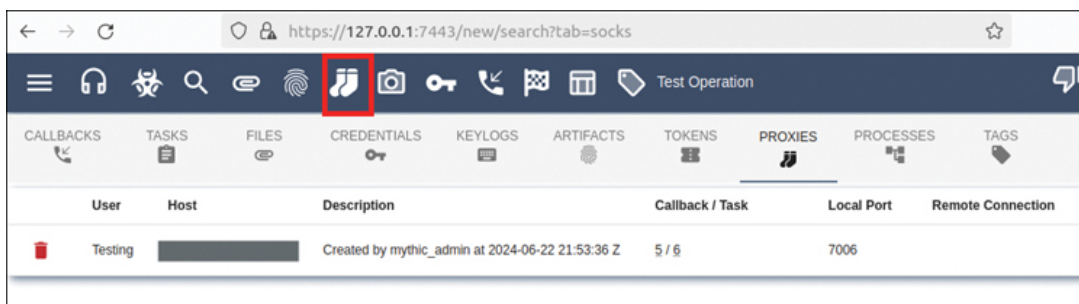


Figure 3-20

Mythic SOCKS Proxies

To test the proxy, navigate to the C2 server and grab your IP through ipconfig.io. For the example in [Figure 3-21](#), the curl without proxychains4 versus with proxychains4 showcases how the traffic is properly routing through the victim's item when using proxychains4.

```

azureuser@EndServer:/opt$ sudo curl ipconfig.io
3.14 Azure Instance IP
azureuser@EndServer:/opt$ sudo proxychains4 curl ipconfig.io
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.14
[proxychains] Strict chain ... 127.0.0.1:7006 ... ipconfig.io:80 ... OK
.255 Proxying Traffic Through Victim's IP

```

Figure 3-21

Testing Proxy

Tip

For a real operation, use an operator box that is separate from the C2 server and set up proxychains4 on that host. In the /etc/proxychains4.conf file, you add the **socks5** line to the file similarly to what is shown in **Example 3-12**, but replace 127.0.0.1 with the public or private IP of the C2 server, depending on where your operator box is hosted. Then you would need to open that specific port to your operator box from your C2 box through firewall rules. This way, you will avoid any attribution to your C2 server if the IP is leaked.

Using a SOCKS proxy is extremely useful for operations; the reason is that you can shy away from executing anything directly on the agent from an OPSEC perspective. Multiple tools can be proxied through a SOCKS proxy, including

- Certipy (<https://github.com/ly4k/Certipy>)
- SCCMHunter (<https://github.com/garrettfoster13/sccmhunter>)
- Impacket (<https://github.com/fortra/impacket>)
- Ldapper (<https://github.com/Synzack/Ldapper>)
- Vibe (<https://github.com/Tylous/Vibe>)

We will dive into tools you'd leverage with the proxy after initial access is obtained in **Chapter 4**.

Another notable area of Mythic from a documentation perspective is the ability to map tasks executed during an operation back to the MITRE ATT&CK matrix using the reporting option of Mythic. You can access this option through the Column icon or at <https://127.0.0.1:7443/new/mitre>.

Once you are on the page, you can fetch all the possible commands from your agents and tasks mapped back to MITRE through the **Actions** tab, as illustrated in **Figure 3-22**. Possible commands are those built into your agent but not necessarily executed tasks.

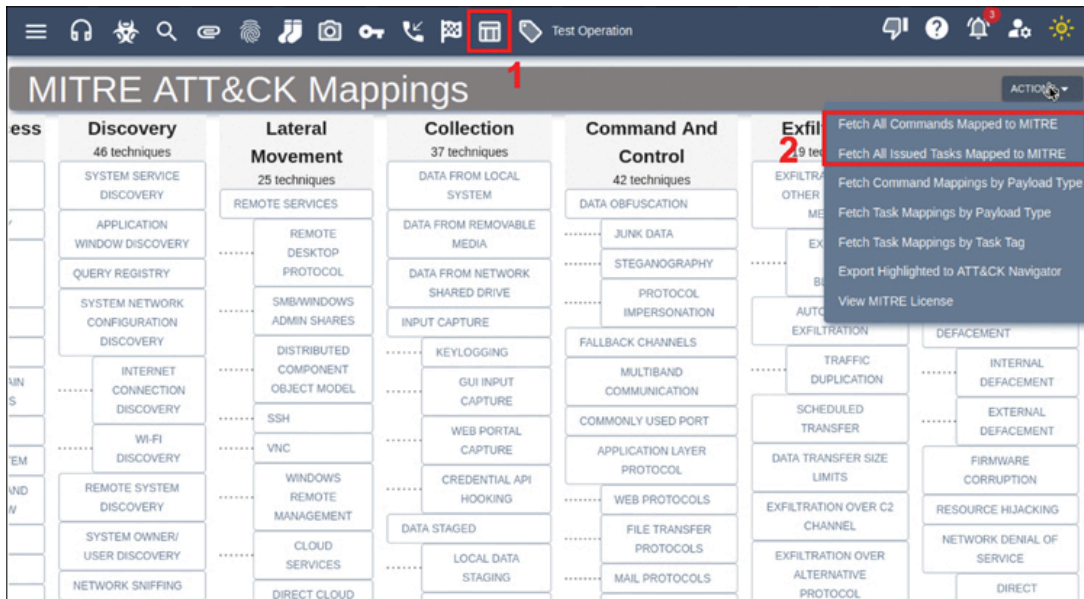


Figure 3-22

MITRE ATT&CK Mappings

For the Fetch All Issued Tasks Mapped to MITRE action, you will see the mappings appear in red showing where they map back to. In this example, shown in **Figure 3-23**, only the Scheduled Transfer mapping appears for the sleep options we used previously.

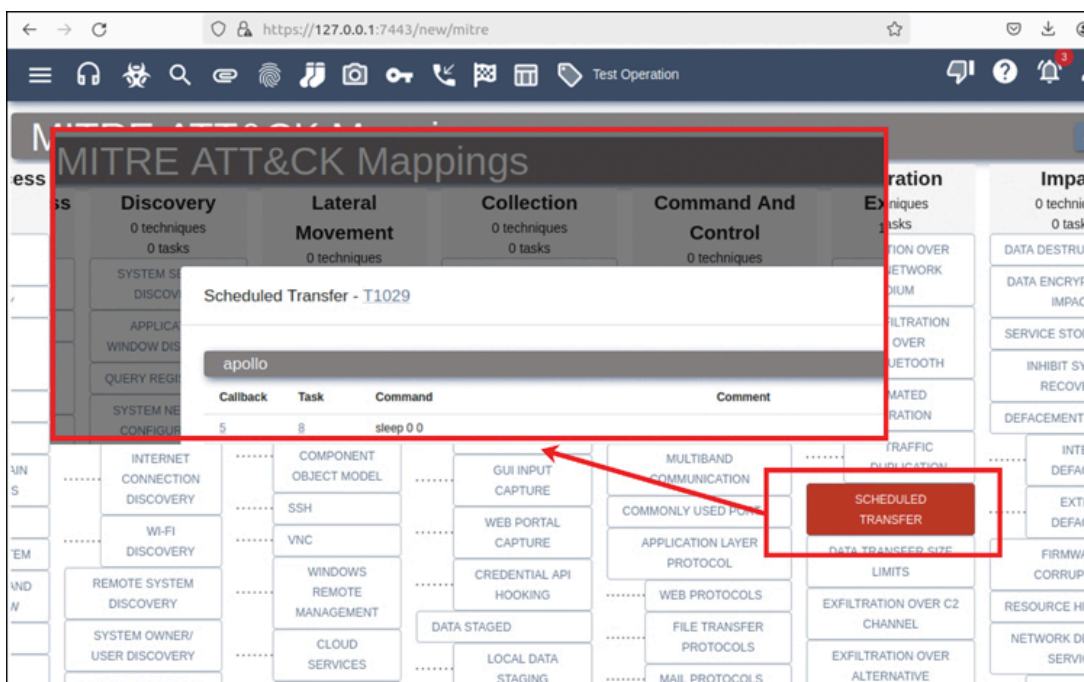


Figure 3-23

Fetch All Issued Tasks Mapped to MITRE Option

To get an idea of mappings associated with the agents, you can select the **Fetch All Commands Mapped to MITRE** action. It is intended to show you only the *potential* mappings based off commands built into the agent you are using, as illustrated in **Figure 3-24**.

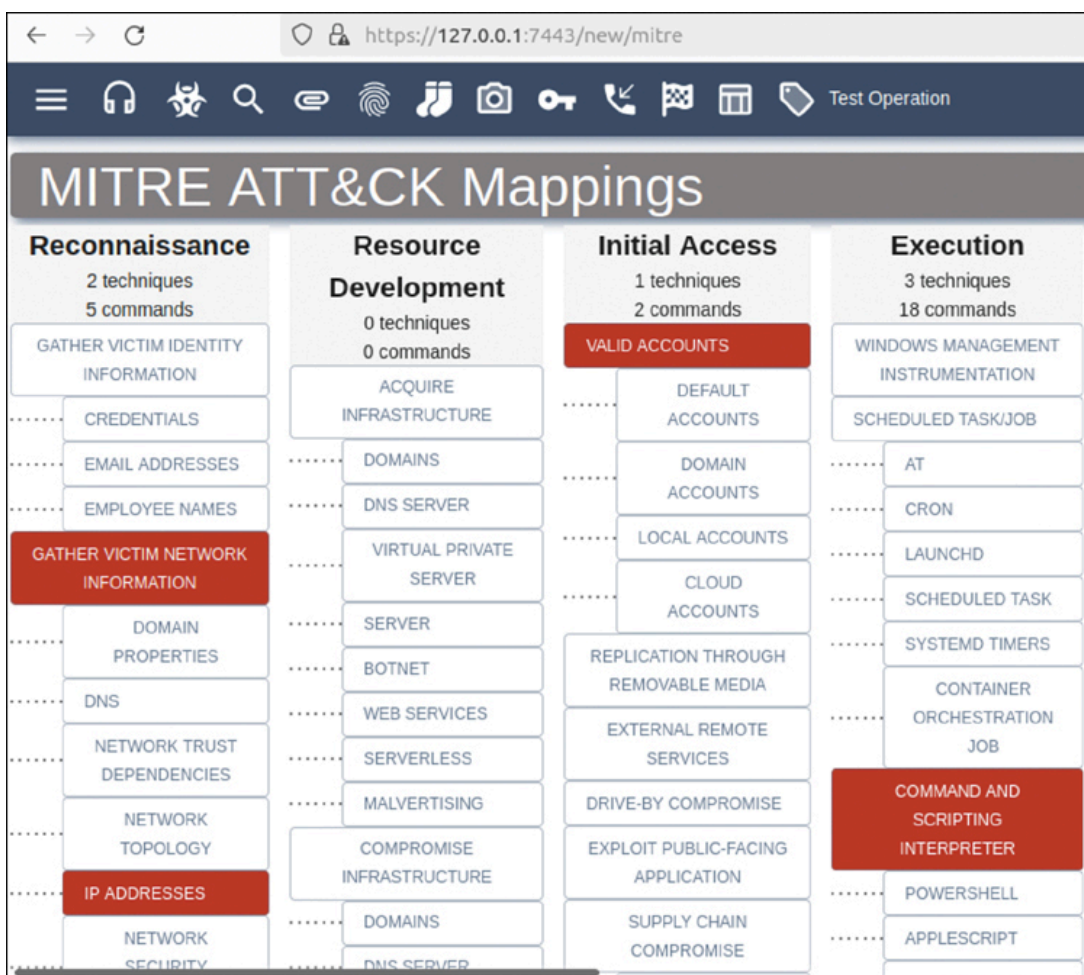


Figure 3-24

Possible Commands from Apollo Agent Mapped to MITRE

In **Figure 3-25**, the Apollo agent has an **ifconfig** command option, which maps back to T1590.005–Gather Victim Network Information: IP Addresses.

IP Addresses - T1590.005

apollo

Command

Documentation

ifconfig

DOCS

CLOSE

Figure 3-25

Command Mapped to T1590.005

To generate a report, navigate to the flag or manually browse to the following URL: <https://127.0.0.1:7443/new/reporting>. You can specify the output to be HTML or JSON. For this example (see [Figure 3-26](#) and [Figure 3-27](#)), we will generate an HTML report. It is best to also include the MITRE coverage options as well. If needed, you can opt to exclude specific hostnames or users.

← → ↻ https://127.0.0.1:7443/new/reporting ☆

☰ 🔊 🔍 🔗 🖨️ 🔑 📞 🚩 🗑️ Test Operation

Mythic Report Generation

Report Option	Selected Values
Output Format	html
Include MITRE ATT&CK Coverage Per Task	☒
Include MITRE ATT&CK Coverage Overview	☒
Exclude Users	Excluded Usernames
Exclude Callbacks With Matching Values	Exclude Hosts
	Excluded Hostnames
	Exclude IDs
	Excluded Callback IDs

GENERATE

Figure 3-26

Mythic Report Creation

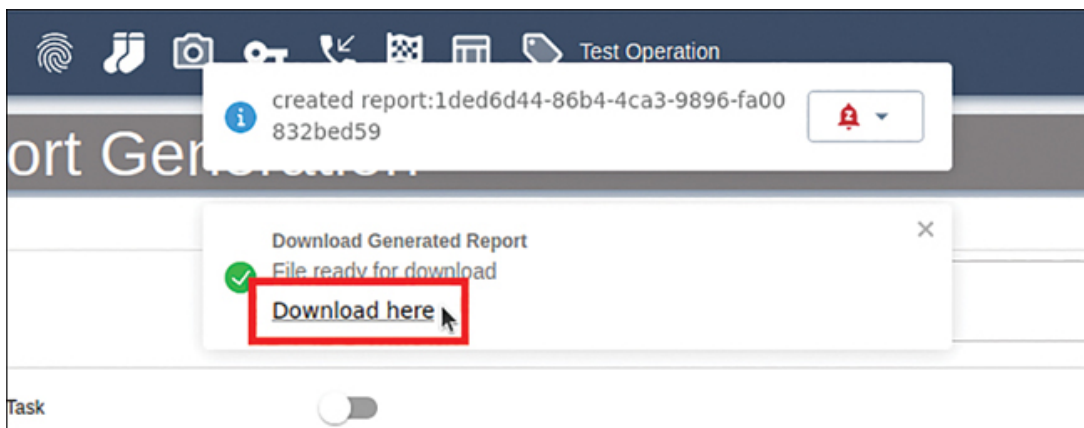


Figure 3-27

Downloading a Mythic Report

Once the report is downloaded, you have detailed information on the operation metrics, callback, MITRE mappings, and artifacts, as illustrated in **Figure 3-28**.



1. Assigned Operators

The following table lists out all of the operators assigned to the operation and their roles in the assessment.

Operator	Role
mythic_admin	lead

2. Operation Metrics

The following table lists out some metrics for the operation.

Metric	Value
Compromised Users	██████████ testing
Compromised Hosts	██████████
Domains Accessed	██████████
Total Callbacks	5
Total High Integrity Callbacks	0
Total Tasks Issued	8

3.1 New Callback 1

Callback Info	Value
User	██████████ Testing
Host	██████████
PID	5664
IPs	██████████ 236
External IP	
Process Name	apollo
Description	Created by mythic_admin at 2024-06-16 22:33:03 Z
OS/Arch	Windows 10 Enterprise 2009 6.2.9200.0/x64
Domain	██████████
Integrity Level	MEDIUM
Initial Checkin	2024-06-16 23:05:09 Z
Last Checkin	2024-06-16 23:08:28 Z
C2 Channels	http

Figure 3-28

Mythic Report Output

The MITRE mapping in the report shown in [Figure 3-29](#) points back to the example shown in [Figure 3-23](#).

Start: 2024-06-22 23:02:09 Z End : 2024-06-22 23:02:09 Z sleep 0 0

T1029

4. MITRE ATT&CK Overview

The following section gives an overview of the amount of coverage for MITRE ATT&CK across all tasks issued.

ATT&CK	Count	Name
T1029	2	Scheduled Transfer

LICENSE The MITRE Corporation (MITRE) hereby grants you a non-exclusive, royalty-free license to use ATT&CK™ for research, development, and commercial purposes. Any copy you make for such purposes is authorized provided that you reproduce MITRE's copyright designation and this license in any such copy. "© 2018 The MITRE Corporation. This work is reproduced and distributed with the permission of The MITRE Corporation." DISCLAIMERS MITRE does not claim ATT&CK enumerates all possibilities for the types of actions and behaviors documented as part of its adversary model and framework of techniques. Using the information contained within ATT&CK to address or cover full categories of techniques will not guarantee full defensive coverage as there may be undisclosed techniques or variations on existing techniques not documented by ATT&CK. ALL DOCUMENTS AND THE INFORMATION CONTAINED THEREIN ARE PROVIDED ON AN "AS IS" BASIS AND THE CONTRIBUTOR, THE ORGANIZATION HE/SHE REPRESENTS OR IS SPONSORED BY (IF ANY), THE MITRE CORPORATION, ITS BOARD OF TRUSTEES, OFFICERS, AGENTS, AND EMPLOYEES, DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION THEREIN WILL NOT INFRINGE ANY RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE

Figure 3-29

Mythic Report MITRE ATT&CK Mappings

Test Your Skills

Multiple-Choice Questions

1. Which redirector is the most sophisticated for red team operations?

1. iptables
2. socat
3. Reverse Proxy
4. CDN with Reverse Proxy using Rewrites

2. What is the name of the Azure service tag used to allow only Azure's source IPs to access your CDN?

1. AzureFrontDoor.Backend
2. AzureBackEnd.FrontDoor
3. AzureSourceIPs
4. AzureFrontDoor

3. What is the overall purpose of a C2 channel?

1. To send data, receive data, and maintain persistent access to an agent
2. To send phishing emails
3. To provide persistent access
4. To receive data

4. What is the command syntax to turn on the SOCKS5 proxy through your Mythic agent?

1. socks-begin 8080
2. socks 8080
3. socks start 8080
4. socks-start 8080

5. Which host do you put a CDN in front of?

1. External web server
2. C2 server
3. Operator machine

4. Victim's machine

6. Which domain and IP do you add to the redirector's /etc/hosts file?

1. Redirector IP and redirector domain
2. C2 IP and C2 domain
3. Redirector IP and C2 domain
4. C2 IP and redirector domain

7. Which C2 is publicly available?

1. Nighthawk
2. Cobalt Strike
3. Mythic
4. None of these options are correct.

8. Which tool can be used to generate SSL certificates with Let's Encrypt?

1. certbot
2. certutil
3. cloudflare
4. smbclient

9. Which file(s) from Let's Encrypt will be imported into Mythic's C2 profile options?

1. cert.pem
2. fullchain.pem and cert.pem
3. chain.pem
4. fullchain.pem and privkey.pem

10. What is the purpose of the Callback Jitter option in your Mythic C2 profile?

1. To specify how often the agent calls back in seconds
2. To add randomness to the callback to make each check-in vary
3. To use as the encryption method for the traffic
4. To customize the GET/POST messages

